

Relatório: Produtor-Consumidor com Buffer Limitado - Threads e Semáforos

Link do repositório: [rafaelapnunes/threads-semaforos-so](https://github.com/rafaelapnunes/threads-semaforos-so)

1. Introdução

Sistemas operacionais modernos executam múltiplas threads de forma concorrente sobre um número limitado de núcleos. Para isso, o escalonador recorre à *preempção*: interrompe uma thread em qualquer ponto (inclusive no meio de uma sequência de instruções que, para quem escreveu o código, parecia "uma operação só") e dá a vez a outra. Quando duas threads compartilham dados mutáveis e ao menos uma delas escreve, a ordem imprevisível dessas trocas de contexto pode produzir resultados incorretos e não determinísticos: a *condição de corrida*.

O enunciado do trabalho pede que se prove, em execução, que o código não garante exclusão mútua e que ela passa a ser garantida com o uso de semáforos, medindo também o tempo de execução em cada condição. Este relatório atende a isso produzindo evidência executável de dois fatos distintos, e não de um só:

1. que o acesso concorrente a um buffer compartilhado **sem exclusão mútua** corrompe os dados, mesmo quando todo o resto do programa está correto; e
2. que **exclusão mútua sozinha não basta** para este problema.

O problema escolhido foi o produtor-consumidor com buffer limitado, implementado em Python 3 com `threading.Thread` e `threading.Semaphore`.

2. O problema escolhido e por que ele pede semáforos

Um conjunto de threads *produtoras* gera itens e os deposita em um buffer circular de capacidade fixa; um conjunto de threads *consumidoras* retira esses itens do mesmo buffer. O problema foi proposto por Edsger Dijkstra junto com os próprios semáforos e suas operações indivisíveis **P** (`acquire`) e **V** (`release`).

A escolha se justifica porque o problema exige **duas** formas de sincronização ao mesmo tempo, e não uma:

Dimensão	O que exige	Primitiva adequada
Exclusão mútua	duas threads não podem manipular o vetor e os índices ao mesmo tempo	semáforo binário (<code>mutex</code>)
Sincronização por condição	produtor precisa <i>esperar</i> se o buffer está cheio; consumidor, se está vazio	semáforos contadores (<code>vazio</code> , <code>cheio</code>)

É exatamente esse segundo requisito que torna o problema adequado ao enunciado, que pede explicitamente um problema cuja viabilidade de correteude ocorra através de semáforos. Um mutex simples resolve a primeira dimensão e é incapaz de expressar a segunda, porque não tem contagem, não sabe representar "restam 7 posições livres" nem bloquear a thread até que esse número seja maior que zero. O semáforo contador resolve as duas coisas com uma única primitiva, e é isso que a variante A deste experimento demonstra na prática, por contraste.

3. Desenho experimental: três variantes, não duas

Este é o ponto central da metodologia. O desenho intuitivo do experimento seria comparar duas versões ("sem nada" contra "com semáforos") e mostrar que a primeira falha. **Esse desenho não prova nada sobre exclusão mútua**, porque as duas versões diferem em *duas* coisas ao mesmo tempo: a versão insegura tipicamente também perde o controle de fluxo (estoura o buffer ou recusa itens quando ele está cheio). Quando ela falha, não há como atribuir a falha à ausência de exclusão mútua.

Para isolar cada causa, o experimento implementa **três** variantes do mesmo buffer, com a mesma interface (*inserir/remover*) e a mesma estrutura interna, removendo **um** mecanismo por vez:

Variante	Exclusão mútua	Controle de fluxo	Falha esperada
A — <i>BufferSoMutex</i>	sim (<i>mutex</i>)	não (erro em vez de espera)	itens recusados e descartados; nenhum dado corrompido
B — <i>BufferSemMutex</i>	não	sim (<i>vazio/cheio</i>)	dados corrompidos; nenhum item recusado
C — <i>BufferCompleto</i>	sim (<i>mutex</i>)	sim (<i>vazio/cheio</i>)	nenhuma

As variantes B e C são a comparação decisiva: elas herdam da mesma classe base, têm o mesmo controle de fluxo e diferem **exclusivamente** pela presença do *mutex* em volta da seção crítica. Qualquer divergência entre as duas é, por construção, atribuível à exclusão mútua e a nada mais. A variante A cobre o outro lado do argumento: ela tem exclusão mútua perfeita e ainda assim falha, o que refuta a ideia de que basta um mutex.

4. Implementação

O código está em *src/*, com uma responsabilidade por módulo:

Módulo	Conteúdo
<i>config.py</i>	parâmetros do experimento (capacidade, número de threads, itens, rodadas) e

	caminhos de saída
<code>buffers.py</code>	classe base do buffer circular e as três variantes
<code>produtor.py</code>	thread produtora
<code>consumidor.py</code>	thread consumidora
<code>verificacao.py</code>	apuração de uma rodada e métricas de corretude
<code>experimento.py</code>	execução das variantes e geração dos CSVs

Como todos os parâmetros vêm de `config.py`, as três variantes são necessariamente executadas sob carga idêntica: não há como uma delas rodar com um número diferente de threads ou de itens.

4.1 A seção crítica, comum às três variantes

A classe base `BufferCircular` concentra o estado compartilhado (vetor pré-alocado e os dois índices) e as duas seções críticas. As subclasses só decidem o que colocar em volta delas:

```
def _escrever(self, item):
    """Secao critica de escrita: le o indice, grava, avanca o
    indice."""
    indice = self._indice_escrita
    janela_de_preempcao()
    self._vetor[indice] = item
    self._indice_escrita = (indice + 1) % self.capacidade
```

`janela_de_preempcao()` é `time.sleep(0)`, que força o interpretador a considerar uma troca de thread naquele ponto. A justificativa é metodológica e está discutida na seção 8: a corrida existe com ou sem essa chamada, mas sob o GIL do CPython uma seção crítica de poucos bytescodes quase nunca é preemptada no ponto exato, e o experimento mediria zero sem que isso significasse ausência de erro. A chamada é executada pelas **três** variantes, no mesmo ponto, o que preserva a validade da comparação.

4.2 As três variantes

```
class BufferSoMutex(BufferCircular):          # variante A
    def inserir(self, item):
        self._mutex.acquire()
        try:
            if self._quantidade >= self.capacidade:
                raise BufferCheio("buffer cheio") # nao espera:
recusa
            self._escrever(item)
```

```

        self._quantidade += 1
    finally:
        self._mutex.release()

class BufferSemMutex(BufferComContadores):    # variante B
    def inserir(self, item):
        self._vazio.acquire()
        self._escrever(item)                # secao critica desprotegida
        self._cheio.release()

class BufferCompleto(BufferComContadores):    # variante C
    def inserir(self, item):
        self._vazio.acquire()                # espera uma posicao livre
        self._mutex.acquire()                # entra na secao critica
        self._escrever(item)
        self._mutex.release()
        self._cheio.release()                # sinaliza um item pronto

```

`BufferComContadores` é a base compartilhada por B e C e contém apenas os dois semáforos contadores:

```

self._vazio = threading.Semaphore(capacidade)    # posicoes livres
self._cheio = threading.Semaphore(0)             # itens prontos

```

`remove()` é simétrico em todas as variantes, com `cheio/vazio` trocados de papel.

4.3 Instrumentação sem novas corridas

Cada `Produtor` insere itens (`id_produto`, `sequencial`), **únicos no experimento inteiro**, e guarda o que conseguiu inserir em um atributo da própria thread; cada `Consumidor` guarda o que recebeu. Nenhum contador é compartilhado entre threads e nenhum lock novo é usado: a apuração lê esses atributos apenas depois do `join()`. Isso é deliberado, pois uma instrumentação compartilhada acrescentaria uma corrida própria ao experimento e contaminaria a medição da corrida que se quer observar.

Nenhuma estrutura de alto nível já sincronizada (`queue.Queue`, `Condition`, `Lock`) foi utilizada: todo o controle de concorrência é feito explicitamente com `threading.Semaphore`.

5. Critério de corretude

A unicidade dos itens permite verificar a execução comparando o *multiconjunto* de itens inseridos com o de itens entregues. Numa execução correta os dois coincidem, a menos do que tenha sobrado no buffer. As métricas apuradas em `verificacao.py` são:

Métrica	Significado	Aponta para
<code>falhas_insercao</code>	inserções recusadas (buffer cheio)	falta de controle de fluxo

<code>falhas_remocao</code>	remoções recusadas (buffer vazio)	falta de controle de fluxo
<code>entregas_duplicadas</code>	mesmo item entregue a mais de um consumidor	falta de exclusão mútua
<code>leituras_invalidas</code>	leitura de posição nunca escrita (<code>None</code>)	falta de exclusão mútua
<code>itens_perdidos</code>	item inserido que nunca foi entregue nem sobrou no buffer (foi sobrescrito)	falta de exclusão mútua
<code>threads_travadas</code>	threads ainda vivas depois do <code>join()</code> com timeout	<i>deadlock</i> ou <i>livelock</i>

A carga é balanceada de propósito (o total que os consumidores tentam remover é igual ao total que os produtores tentam inserir), de modo que, numa execução correta, o buffer termina vazio e qualquer sobra já é sintoma de erro. Uma rodada é considerada correta apenas se todas as seis métricas forem zero.

6. Metodologia

Parâmetros (em `config.py`, idênticos para as três variantes):

- capacidade do buffer: **10** posições;
- **4** threads produtoras × **1000** itens = **4000** itens a inserir;
- **4** threads consumidoras × **1000** itens = **4000** itens a remover;
- **10** rodadas independentes por variante, 30 no total (buffer e threads recriados a cada rodada);
- `timeout` de 30 s no `join()`, como rede de segurança contra travamento.

O tempo de cada rodada é medido com `time.perf_counter()`, do `start()` da primeira thread ao `join()` da última. Não há nenhum `sleep` de duração arbitrária no laço das threads, apenas a cessão de vez descrita em 4.1, de custo desprezível. Isso é relevante: um atraso artificial de milissegundos por iteração dominaria a medição e tornaria a comparação de tempo entre as variantes um artefato do próprio atraso, e não do custo de sincronização.

Ambiente de execução:

- Windows 11 (Windows-11-10.0.26200-SP0);
- CPython **3.13.9**, intervalo de troca de threads padrão (5 ms);
- Intel64 Family 6 Model 140 (Intel Core de 11ª geração), 8 núcleos lógicos.

Execução e saídas:

```
python src/experimento.py           # todas as variantes
python src/experimento.py --variantes B C  # apenas a comparação decisiva
python src/experimento.py --rodadas 3      # menos rodadas
```

- `resultados/rodadas.csv`: uma linha por rodada, com as métricas da seção 5;
- `resultados/resumo.csv`: uma linha por variante, com médias e taxa de falhas.

7. Resultados

Resumo de 10 rodadas por variante, transcrito de `resultados/resumo.csv`:

Variante	Tempo médio (s)	Desvio (s)	Rodadas corretas	Recusas de inserção (méd.)	Recusas de remoção (méd.)	Entregas duplicadas (méd.)	Itens perdidos (méd.)
A só mutex	0,0823	0,0132	0/10	1286,3	1288,4	0,0	0,0
B sem mutex	0,0690	0,0011	0/10	0,0	0,0	2076,3	2076,3
C completa	0,1285	0,0034	10/10	0,0	0,0	0,0	0,0

Dispersão rodada a rodada (de `resultados/rodadas.csv`):

Variante	Itens inseridos (de 4000)	Recusas de inserção	Entregas duplicadas	Itens perdidos
A	1269 – 3329	671 – 2731	0	0
B	4000 – 4000	0	2053 – 2102	2053 – 2102
C	4000 – 4000	0	0	0

Duas métricas ficaram em zero nas 30 rodadas. `threads_travadas` nunca disparou: nenhuma das variantes chegou a travar, o que era esperado, já que as operações `acquire/release` permanecem individualmente atômicas mesmo na variante B. `leituras_invalidas` também ficou em zero, e por um motivo que vale registrar: essa métrica só detecta a leitura de uma posição *nunca* escrita, o que na prática ocorreria apenas na primeira volta do buffer circular; depois disso toda posição já contém algum item antigo, e uma leitura fora de hora devolve um item real repetido, capturado por `entregas_duplicadas` em vez de `None`.

Experimento de controle, fora do fluxo principal: desativando `janela_de_preempcao()` (tornando-a um `pass`), a variante B passou a completar 4000 inserções e 4000 entregas **sem nenhuma divergência** em três execuções consecutivas.

8. Discussão

A variante A prova que exclusão mútua não é suficiente. Ela tem um semáforo binário serializando toda a seção crítica, e o efeito aparece nos números: zero entregas duplicadas, zero leituras inválidas, zero itens perdidos em 10 rodadas. E mesmo assim ela falhou em 10

de 10 rodadas, recusando em média 1286 das 4000 inserções (32%) e 1288 das 4000 remoções. A causa é a outra dimensão do problema: sem contagem, o buffer não tem como fazer o produtor *esperar* por uma posição livre, então o item é simplesmente descartado. Um mutex não resolve isso porque não é contador.

A variante B prova que exclusão mútua é necessária. Com os dois semáforos contadores, o controle de fluxo fica perfeito: 4000 inserções e 4000 remoções em todas as rodadas, sem uma única recusa e sem nenhuma thread travada. Ainda assim, **em média 2076 das 4000 entregas foram duplicadas (52%)** e outros 2076 itens foram perdidos. O mecanismo é o esperado quando a seção crítica fica desprotegida:

- dois produtores leem o mesmo `_indice_escrita` antes de qualquer um deles avançá-lo; ambos gravam na mesma posição e o item do primeiro é **sobrescrito**: o item entrou no buffer, foi contabilizado pelo semáforo `cheio`, e nunca chegará a um consumidor;
- dois consumidores leem o mesmo `_indice_leitura`; ambos recebem o **mesmo item**, que é entregue duas vezes.

Os dois números coincidem por aritmética, e não por coincidência: como `vazio/cheio` mantém o total de entregas exatamente igual ao total de inserções, cada entrega duplicada ocupa o lugar de um item que ficou sem ser entregue. É a assinatura de uma violação de exclusão mútua com contabilidade de fluxo intacta.

Vale notar o que **não** aparece em B: nenhuma recusa, nenhum travamento, nenhuma exceção. As operações `acquire/release` continuam corretas mesmo sem o mutex, porque cada uma delas é atômica por si (implementada sobre primitivas do SO, fora do alcance do GIL). O que se perde sem o mutex não é a contabilidade. Um experimento que só olhasse exceções ou totais agregados concluiria, erradamente, que B está correta.

A variante C é correta e o custo é mensurável. 10 rodadas corretas em 10. A comparação de tempo justa é C contra B, que só diferem pelo mutex: 0,1285 s contra 0,0690 s, ou seja, **+86% de tempo** de execução. Esse é o custo real de serializar a seção crítica. O tempo da variante A (0,0823 s) **não é comparável**: ela executou apenas cerca de dois terços do trabalho, porque descartou o resto. Seu desvio padrão é quase quatro vezes o da variante C e doze vezes o da variante B, o que reflete o quanto o resultado dela depende da sorte do escalonamento. É a ilustração prática de que velocidade sem correteude não significa nada.

Sobre o GIL e a janela de preempção. O GIL do CPython garante atomicidade de *bytecodes individuais*, não de sequências. `self._vetor[i] = item` seguido de `self._indice_escrita = (i + 1) % cap` são vários bytecodes, e o interpretador pode trocar de thread entre eles: a corrida é real. Mas ela é *rara* na prática, porque o intervalo padrão de troca é 5 ms (tempo em que uma thread executa milhares de bytecodes) e a chance de a preempção cair exatamente dentro de uma seção crítica de poucas instruções é pequena. O experimento de controle da seção 7 mostra isso quantitativamente: sem a janela explícita, a variante B roda 4000 itens sem nenhuma divergência observável, apesar de não ter proteção nenhuma.

Esse resultado merece ser dito com clareza, porque é o mais importante do ponto de vista prático: **a ausência de erro observado não é evidência de código correto**. O mesmo código, com a mesma falha, executado com uma seção crítica um pouco mais longa, em outra implementação de Python (sem GIL), em outro número de núcleos, ou simplesmente sob outra carga do sistema, corrompe metade das entregas. A `janela_de_preempcao()` não *cria* o defeito da variante B, ela apenas torna reproduzível o pior caso que o escalonador poderia produzir a qualquer momento. É por isso que corretude em programas concorrentes se argumenta pela estrutura do código, e não por teste bem sucedido: bugs de concorrência que não se manifestam em teste continuam existindo em produção.

9. Conclusão

O experimento sustenta as duas afirmações que se propôs a provar, cada uma com uma variante que isola a sua causa. Sem exclusão mútua (variante B), com o controle de fluxo comprovadamente intacto, 52% das entregas saíram corrompidas (itens duplicados e itens sobrescritos) em 100% das rodadas. Com exclusão mútua mas sem semáforos contadores (variante A), o estado interno nunca se corrompeu e ainda assim um terço dos itens foi descartado, mostrando que um mutex resolve só metade do problema. A solução clássica de Dijkstra com três semáforos (variante C) executou corretamente em 10 de 10 rodadas, ao custo de 86% de tempo adicional sobre a versão sem o mutex.

A conclusão prática vai além da comparação: o experimento de controle mostrou que a mesma variante B, defeituosa, pode passar por 4000 operações sem apresentar um único erro quando a janela de preempção não é forçada. Em programas concorrentes, portanto, sincronização não é uma otimização a ser adicionada quando o erro aparecer.